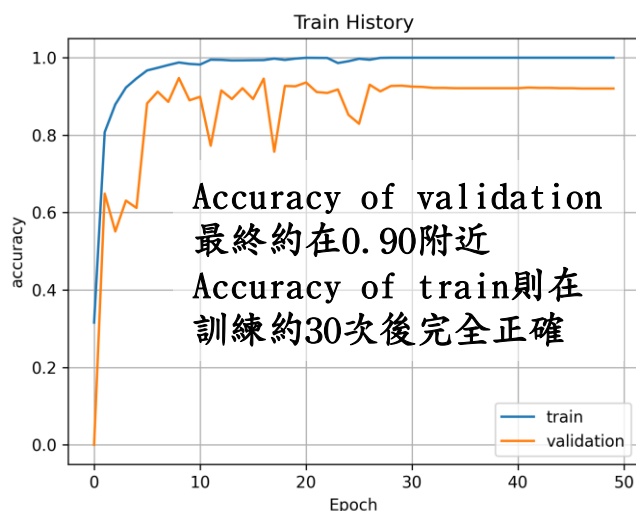
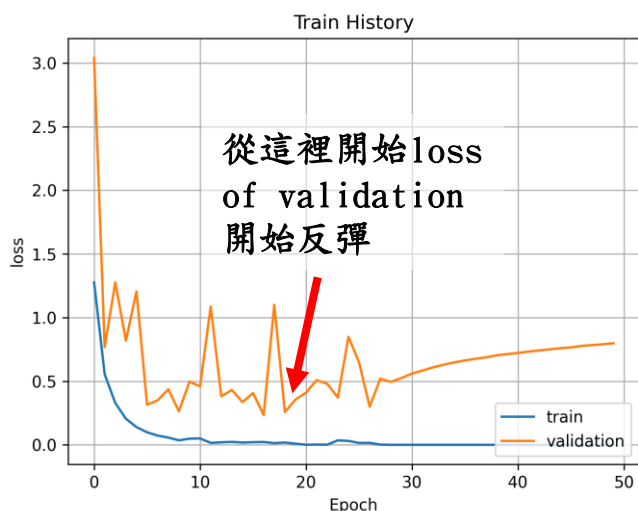


MidTerm Project —

Classification of bolt, locatingpin, nut and washer

大氣4A 108601505 劉忼茗

一、Loss & Accuracy of train and validation data



二、Loss & Accuracy of test data

48/48 [=====] - 8s 175ms/step

48/48 [=====] - 6s 118ms/step - loss: 0.2593 - accuracy: 0.9685

最終測試值其Accuracy與Loss，另有混淆矩陣與數據分析呈現

	precision	recall	f1-score	support
0	0.97	0.90	0.94	381
1	0.90	0.96	0.93	381
2	0.98	0.99	0.98	381
3	1.00	0.99	1.00	381
accuracy			0.96	1524
macro avg	0.96	0.96	0.96	1524
weighted avg	0.96	0.96	0.96	1524

0:bolt, 1:locatingpin, 2:nut, 3:washer

True \ Pred	0	1	2	3
0	343	34	4	0
1	9	367	4	1
2	0	4	377	0
3	0	2	0	379

前頁各分析分別代表：

- ✓ Precision: 精確率就是所有預測正例中，真實也是正例的比率。
- ✓ Recall: 所有正例中，預測也為正例的比率。
- ✓ F1-score: $\frac{2}{\frac{1}{precision} + \frac{1}{recall}}$

可以分辨出bolt、locatingpin的預測精確度與Recall值較差，左圖的混淆矩陣也可看出bolt有較多的預測錯誤，將bolt預測為locatingpin。

三、期末專題實作過程

```
import zipfile

#import image data from google drive
!gdown --id '1qCHfycy91EyUFzilBvxu8hYVYp0l6jCh' --output Midterm_dataset.zip
```

```
#解壓縮zip檔到temp
local_zip = 'Midterm_dataset.zip'
zip_ref = zipfile.ZipFile(local_zip, 'r')

#解壓縮後的目標文件夾(自動創建)
zip_ref.extractall('temp')
zip_ref.close()
```

下載資料，並且解壓縮檔案，將其存入temp的資料夾。

資料前處理(含匯入圖片與統一圖片格式)

嘗試使用過兩種方法，一種是最後使用的，透過迴圈與os.listdir[列出此資料夾內的所有目錄]等指令，讀取資料夾中的圖片，再透過cv2.imread[讀取圖片]、cv2.cvtColor[調整圖片為灰階]、cv2.resize[調整圖片大小]，做圖片的前處理，並透過np.append指令，將所有圖片放進image集合與對應的label集合。

```
# cv讀照片，並resize，且調整原來顏色為RGB，轉為灰階
image = cv2.imread(img_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
image = cv2.resize(image, IMAGE_SIZE)
```

```
# 轉為numpy array的形式，圖片dtype為浮點數，label為整數
images = np.array(images, dtype = 'float32')
labels = np.array(labels, dtype = 'int32')
```

```
#將兩個資料夾的所有資料append起來
output.append((images, labels))
return output
```

```
#個別存取training data and testing data
(x_train,y_train),(x_test,y_test) = load_data()

#x_陣列轉換為四維矩陣，dim0:輸入照片總張數，dim1,2:為image_size，dim3:表灰階僅1，若為RGB則為3
x_train = x_train.reshape(x_train.shape[0],224,224,1).astype('float32')
x_test = x_test.reshape(x_test.shape[0],224,224,1).astype('float32')
```

```
#標準化 - >因為有0-255個色階
x_train = x_train/255
x_test = x_test/255

#將原有的向量轉換為one-hot code，以便進行模型訓練
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

為了後續處理資料方便，將原先的集合轉為numpy.array，並設定資料類型分別為浮點數與整數。最後append已經跑完迴圈的image與label矩陣為output矩陣。注意，此處的output(return值)會輸出train與test兩者，不會放進同一個矩陣。(詳細程式碼見壓縮檔)

透過load_data()，輸出會使用的train與test資料，並進行資料正規化，詳細調整如圖之說明。

第二種方法是透過tensorflow.keras圖片資料處理參數ImageDataGenerator，直接將資料夾內的圖片輸入。

```
train_datagen = ImageDataGenerator(
    rescale=1./255, # 圖片像素值為0-255，此處都乘以1/255，調整到0-1之間
)
train_generator = train_datagen.flow_from_directory( # 從文件夾中產生數據流
    directory="/content/temp/MidTerm-Dataset/train", # 訓練集圖片的文件夾
    target_size=(224, 224), # 調整後每張圖片的大小
    class_mode="sparse",
    batch_size=32,
)
```

此參數可直接進入資料夾內做分類，並直接輸出含label之dataset，也可在參數內進行資料正規化。

後來沒使用的原因是，這個方法後面無法使用validation_split的參數語法，需要事先分割資料為train & validation，因此雖然這樣可以使程式碼簡潔，但最後還是使用第一種方法做處理，為後續model training能直接透過validation_split做資料切割。

Model Building

事先搜尋過幾個經典的CNN模型後，最後決定採用VGG16。訓練層數不算多(電腦不會爆掉的許可範圍)，但能取得不錯的訓練成果。

```
#建立模型，此處使用經典VGG16作為建模型態，共有五層，無dropout
def VGG16(input_shape=(224, 224, 1), num_classes=4):
    img_input = Input(shape=input_shape)
    # img_input = data_augmentation(img_input1)

    #block 1
    x = Conv2D(filters=64, kernel_size=(3, 3),
               activation='relu', padding='same')(img_input)
    x = Conv2D(filters=64, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)

    #block 2
    x = Conv2D(filters=128, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=128, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)

    #block 3
    x = Conv2D(filters=256, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=256, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=256, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)

    #block 4
    x = Conv2D(filters=512, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=512, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=512, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)

    #block 5
    x = Conv2D(filters=512, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=512, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = Conv2D(filters=512, kernel_size=(3, 3),
               activation='relu', padding='same')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)
```

VGGNet為較深層的網路，特點為重複採用同一組基礎模型，並使用多個小卷積(3x3)來取代中大型卷積核(5x5)。其中VGG Block為不同數量的3x3卷積層與2x2的池化層組成。VGGNet有不同的結構，如VGG11、VGG13、VGG16等等，差異為網路的層數。VGG16主要使用了五個卷積層與三個全連接層，前兩個卷積層內含2個基礎模組，後三個卷積層則含3個基礎模組，總合為16層網路。

```
#flatten
x = Flatten()(x)
x = Dense(4096, activation='relu')(x)
x = Dense(4096, activation='relu')(x)
x = Dense(num_classes, activation='softmax')(x)

return Model(inputs=img_input, outputs=x)
```

```
#建立模型，此處使用經典VGG16模型
model = VGG16()
model.summary()
```

完整的model.summary() 資訊可見程式碼詳列。

Compile the model

編譯模型，配置訓練方法時，告知訓練時用的優化、損失函數與準確度評估標準。例如準確度除了”accuracy”，尚有”sparse_accuracy”和”sparse_categorical_accuracy”可輸出。損失函數也有”mse”等可以輸出。

```
from tensorflow.keras.optimizers import Adam

model.compile(
    loss='categorical_crossentropy', #多元分類問題(整數標籤)，故使用此損失函數
    metrics=['accuracy'], #準確率
    optimizer=Adam(learning_rate=0.0001) #lr:學習率 = 0.0001
    #optimizer='adam' #用這種方法也行，但只能使用預設參數(此處無使用)
)
```

Train the model

```
train_history = model.fit(
    x_train,y_train, #輸入INPUT
    epochs=50, #回圈次數
    verbose=1, #顯示詳細訊息
    validation_split=0.2, #驗證資料與訓練資料比例
    use_multiprocessing=True, #每個輸入在每個epochs只使用一次
    batch_size=32 #每個梯度的更新樣本數
)
```

將 x_train, y_train 丟入模型中進行訓練，次數設置為 50 次，validation 設定為 0.2，即有 20% 的 x_train 切割為驗證資料。

```
Output exceeds the size limit. Open the full output data in a text editor
Epoch 1/50
153/153 [=====] - 102s 494ms/step - loss: 1.2777 - accuracy: 0.3185 - val_loss: 2.5508 - val_accuracy: 0.0000e+00
Epoch 2/50
153/153 [=====] - 67s 437ms/step - loss: 1.2694 - accuracy: 0.3058 - val_loss: 2.3972 - val_accuracy: 0.0000e+00
Epoch 3/50
153/153 [=====] - 68s 443ms/step - loss: 0.7770 - accuracy: 0.6591 - val_loss: 1.2557 - val_accuracy: 0.5488
Epoch 4/50
153/153 [=====] - 67s 441ms/step - loss: 0.3547 - accuracy: 0.8695 - val_loss: 0.8014 - val_accuracy: 0.6530
Epoch 5/50
153/153 [=====] - 67s 439ms/step - loss: 0.2176 - accuracy: 0.9183 - val_loss: 1.2382 - val_accuracy: 0.5299
Epoch 6/50
153/153 [=====] - 67s 441ms/step - loss: 0.1380 - accuracy: 0.9491 - val_loss: 0.5840 - val_accuracy: 0.7260
Epoch 7/50
153/153 [=====] - 67s 439ms/step - loss: 0.1055 - accuracy: 0.9627 - val_loss: 0.5396 - val_accuracy: 0.8064
Epoch 8/50
153/153 [=====] - 67s 441ms/step - loss: 0.1042 - accuracy: 0.9633 - val_loss: 0.3025 - val_accuracy: 0.8893
Epoch 9/50
153/153 [=====] - 68s 443ms/step - loss: 0.0495 - accuracy: 0.9826 - val_loss: 0.6738 - val_accuracy: 0.7990
```

注意，訓練時不能放入測試資料，避免測資一起訓練，造成最終分數有高估情形。

Graph

透過matplotlib.pyplot來繪製訓練得到的accuracy與loss。詳細說明如圖中註解：

```
#繪製準確度之圖
import matplotlib.pyplot as plt

plt.plot(train_history.history['accuracy'])
plt.plot(train_history.history['val_accuracy'])
plt.title('Train History') #主標題
plt.ylabel('accuracy') #y軸名稱
plt.xlabel('Epoch') #x軸名稱
plt.legend(['train', 'validation'], loc='lower right') #繪製圖例
plt.grid() #繪製網格
plt.savefig('Train_History_accuracy.png',dpi=300) #存圖，畫值取300dpi

plt.show()
```

Prediction

將x_test資料放入模型中預測，並將輸出結果與y_test做比對，計算準確度與損失函數。

```
#預測模型，放入x_test
prediction=model.predict(x_test)
#classes_x=np.argmax(prediction,axis=1)
#計算損失函數與準確度
score = model.evaluate(x_test,y_test)
```

```
48/48 [=====] - 9s 183ms/step
```

```
48/48 [=====] - 6s 123ms/step - loss: 0.3515 - accuracy: 0.9619
```

Confusion Matrix

```
#建立混淆矩陣，分析模型好壞
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# print(y_test)
#改變y_test,y_pred的矩陣，用以分析模型好壞
y_test = np.argmax(y_test, axis=1)
y_pred = np.argmax(prediction,axis=1)
```

將y_test、prediction變為1D矩陣，以便放入混淆矩陣的function計算，並以視覺化的方式呈現預測結果。完整註解可見程式碼，太長沒有截全圖。

```
#繪製heatmap觀察預測與正確label差異
cm=confusion_matrix(y_test,y_pred)

import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(15,8)) #annot:添加註釋(即方框中數字) fmt:註釋使用格式(整數) linecolor:方框之間線顏色 cmap:
sns.heatmap(cm,square=True,annot=True,fmt='d',linecolor='white',cmap='RdYlBu',linewidths=1.5,cbar=False)
sns.set(font_scale=5) #方格內字型大小設定
plt.xlabel('Pred',fontsize=20)
plt.ylabel('True',fontsize=20)
plt.show()

print(classification_report(y_test,y_pred))
```

End

四、實作過程中額外嘗試－Data Augmentation

```
# #data augmentation資料翻轉
# data_augmentation = keras.Sequential(
#     [
#         #垂直翻轉與水平翻轉
#         tf.keras.layers.experimental.preprocessing.RandomFlip("horizontal"),
#         tf.keras.layers.experimental.preprocessing.RandomFlip("vertical"),
#     ]
# )
```

有嘗試過將原有的train_data進行資料翻轉後(水平與垂直)，再丟進去訓練，但最終訓練出來的結果與沒翻轉的結果差不多，推測是資料量不足或是可以再多加旋轉角度進去，讓訓練更加準確。

48/48 [=====] - 6s 116ms/step
48/48 [=====] - 6s 119ms/step - loss: 0.3259 - accuracy: 0.9600

上圖準確度是有加入資料翻轉後的結果，但感覺上效果不是很好。

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2

這是keras官方給出的各模型檢測數據，VGG16最終validation訓練數值確實是接近90%。

四、參考資料

- ✓ Stackoverflow → debug用
- ✓ <https://keras.io/api/applications/>
- ✓ <https://ithelp.ithome.com.tw/articles/10235805>
- ✓ [容嘆玩Data](#)：深度學習－神經網路
- ✓ <https://keras.io/zh/models/model/>
- ✓ <https://blog.csdn.net/moyul23456789/article/details/83444140>
- ✓ <https://colab.research.google.com/github/yenlung/Deep-Learning-Basics/blob/master/colab02c%20%E7%94%A8%E9%81%B7%E7%A7%BB%E5%AD%B8%E7%BF%92%E6%89%93%E9%80%A0%E5%85%AB%E5%93%A5%E8%BE%A8%E8%AD%98AI.ipynb#scrollTo=xi7FwyUhNLbD>
- ✓ <https://weikaiwei.com/python/ml-confusion-matrix/>
- ✓ https://blog.csdn.net/weixin_44520259/article/details/89917026
- ✓ <https://medium.com/ching-i/%E5%8D%B7%E7%A9%8D%E7%A5%9E%E7%B6%93%E7%B6%B2%E7%B5%A1-cnn-%E7%B6%93%E5%85%B8%E6%A8%A1%E5%9E%8B-lenet-alexnet-vgg-nin-with-pytorch-code-84462d6cf60c>
- ✓ <https://blog.csdn.net/yunfeather/article/details/106461754>
- ✓ 陽明交通大學－深度學習於生醫資料分析課程